

#1: `tensor-on-gpu.c` $\Leftrightarrow$ `torch-tensor.c`

#2: `tensor-operations.c`

$\Downarrow$
`tensor-storage.c` $\rightarrow$ movement / where is the data?

`tensor-slicing.c`

`tensor-elementwise.c` $\rightarrow$ transpose

`tensor-matmul.c`

#3: `tensor-einops.c` $\rightarrow$ mutation of tensor

#4: `tensor-operations-flops.c`

#5: `gradients-basics.c`

#6: `gradients-flops.c`

tensor views are free, but copying takes both additional

memory and CPU compute.

$\Rightarrow$ the bread and butter of deep learning is: matmul.
fundamental.

Coding is the bread and butter of AI research

$\Rightarrow$ `x = torch.ones(4, 8, 16, 32)`
Num depth ~~col~~ row col

`w = torch.ones(32, 2)`
Num ~~row~~ col ~~depth~~

$\Rightarrow y = x @ w$

`assert y.size() == torch.Size([4, 8, 16, 2])`

`einops_einsum()`: is generalized matrix multiplication with good bookkeeping

old way: $X \overset{\text{matrix}}{*} \overset{\text{transposed}}{Y}$
 $\Downarrow$
 $Z = X @ Y.transpose(-2, -1)$

new way: $Z = \text{einsum}(X, Y, \text{"batch seq1 hidden, batch seq2 hidden} \rightarrow \text{batch seq1 seq2"})$

`einops_reduce()`: aggregate a single vector via (mean/median/count)

`einops_rearrange()`: if dimension represents two dimensions and you want to operate on one of them
 $\Downarrow$

Break up total_hidden into two dimensions (heads and hidden1)

$(\text{batch}, \text{seq}, \text{total_hidden}) = (2, 3, 8)$

$\Leftarrow$ multi-head attention style

$\left\{ \begin{array}{l} \text{total_hidden} = \text{heads} * \text{hidden1} \\ 8 = 2 * 4 \end{array} \right.$

$(\text{hidden1}, \text{hidden2}) = (4, 2) \rightarrow \text{linear projection/mapping matrix}$

$\Downarrow$
 $X = \text{rearrange}(X, \text{"... (heads hidden1)"} \rightarrow \text{"... heads hidden1", heads})$

$\Rightarrow$ new $X = (2, 3, 2, 4)$

$X = \text{einsum}(X, W, \text{"... hidden1, hidden1, hidden2"} \rightarrow \text{"... hidden2"})$

$\Rightarrow$ new $X = (2, 3, 2, 2)$ $\Downarrow$ multiply and sum over hidden1, output(..., hidden2)